Object-relational database

Data models in database systems

Lecture 5

Mara Romanovska
Mara.Romanovska@rtu.lv

11.09.2025.

1862
RTU
DATORZINĀTNES UN
INFORMĀCIJAS
TEHNOLOĢIJAS FAKULTĀTE

Where we left off...

- RDBMS – good for many purposes yet not sufficient for
  - Complex data
  - Not possible to store object as a whole (specific schema)

Concepts of object-oriented approach

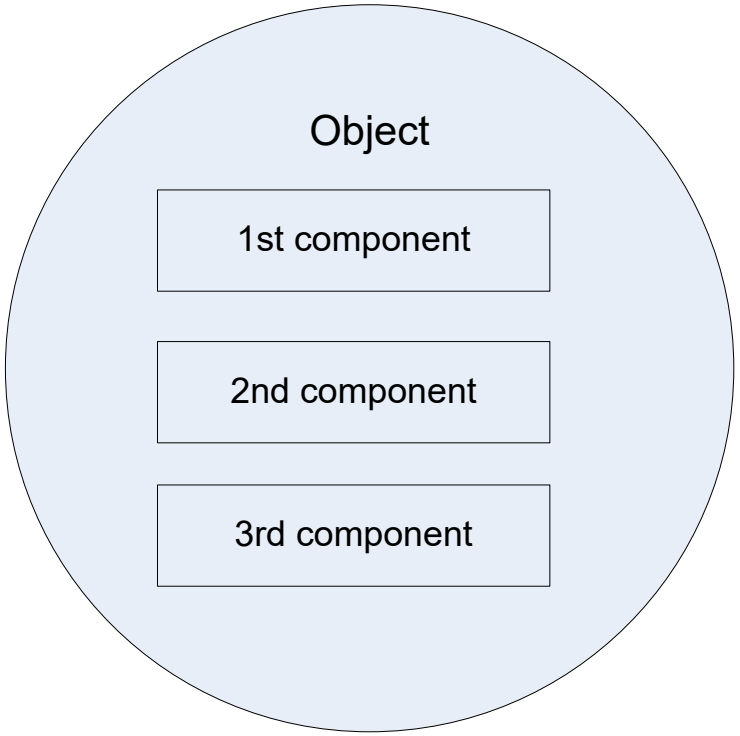
Object-oriented programming (OOP)

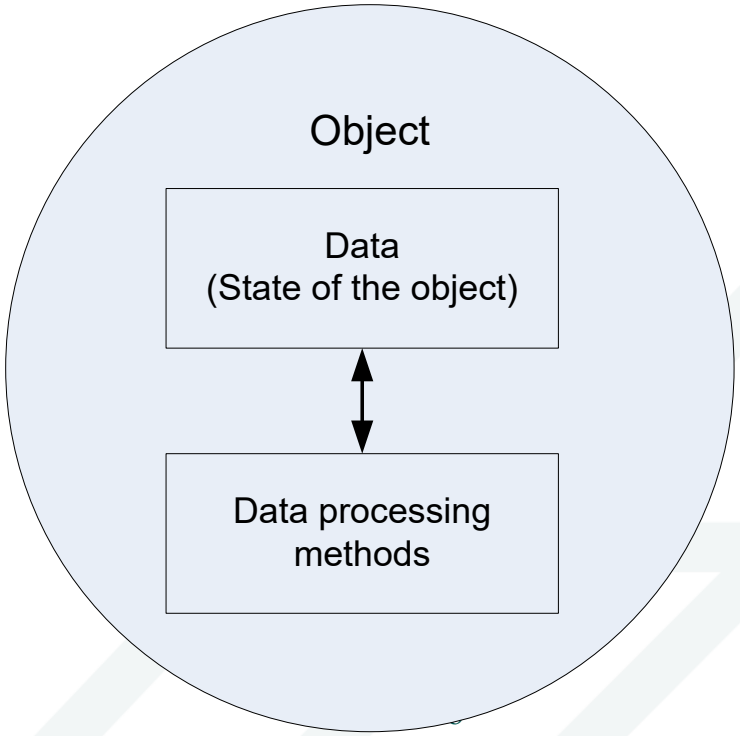
- Nowadays almost all software is developed using the object-oriented approach

- OOP is data centred approach not functions centred
  - The smallest units are defined according to data instead of functions
- The main concept is Object

Interpretations of term “object”

- Term “Object” denotes
  - The unity of components
  - That data and corresponding programs are united

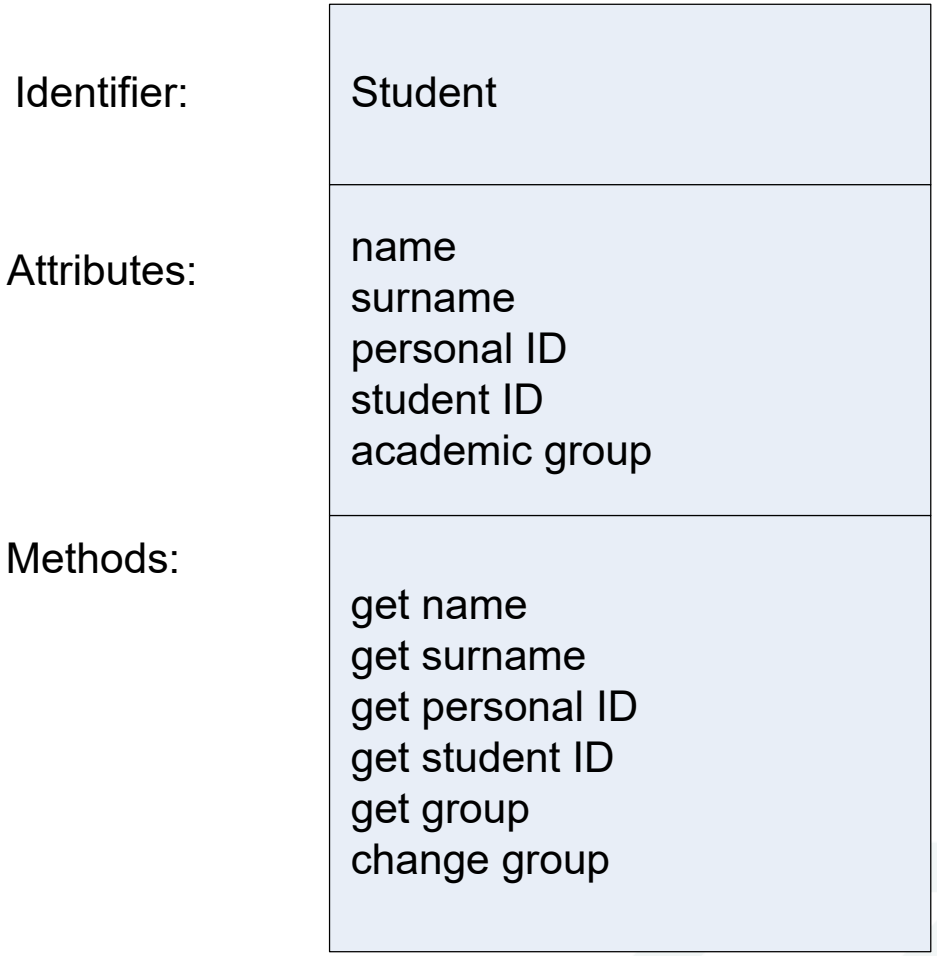




Class

- Class is created for each entity processed by software
  - Class “Student” is created to process student data
- Class consists of
  - Attributes or data
  - Methods or functions that process these data
- Attributes and methods are specified during the class definition

Example of a class



Object

- Object is an instance of some class
- Object contains
  - Its identifier (may be just an address in the memory)
  - Attributes and their values
  - Behaviour of the class (implementation view)
    - What data are processed inside the class?
  - Public interface of the view (user view)
    - How the object can be used from outside?

Example of an object

Identifier:	Student
Atributes:	name: Jānis surname: Kalniņš personal ID: 010101-11111 student ID: 011RDB001 academic group: 1
Methods:	get name get surname get personal ID get student ID get group change group

Basic concepts of OOP

- ***Abstraction.*** User sees an object simpler than it is. The object can be very complex and at the same time have a simple interface
- ***Encapsulation***. Joining data of object's state and the set of the corresponding procedures. It means that not all attributes and methods are available outside it
- ***Inheritance*** allows some class (subclass) to reuse attributes and methods from another class without the need to redefine them

Objects+databases: approaches

- Pure object approach
- Object-relational mapping
- Object-relational databases

Drawbacks of pure object approach

- Eventual consistency
- Relatively slow data querying

- Complexity
- Lack of standards
- Lack of experience
- Lack of support for views
- Lack of adequate security mechanisms

Object-relational databases

Hybrid approach

- Acts as an interface between OO language and RDB
- Belongs to so-called extended RDBMS
- «Best of both worlds»

Advantages of ORDB

- **Objects can encapsulate operations with data.** Objects in ORDB include operations for data processing
- **Objects are efficient.** It is possible to extract and operate with a related set of objects as a whole
- **Objects can represent a *part-whole* relationship.** Objects can include other objects as attributes.
- **Objects are natural**. Type of the object allows to achieve the meaning of the object, i.e. the fact that all parts together make the whole
  - Address can include apartment number, house number, street, city and country
  - If one of these elements is lost, the address is incomplete
  - Relational table can not denote that some elements only together have a meaning

Objects in Oracle

Oracle ORDB

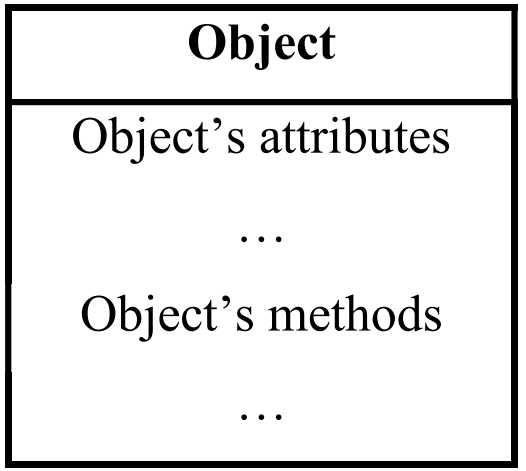
- Oracle supports ORDB starting from the version 8i
- Uses PL/SQL
- **Object type** is used instead of class
- Objects have description – its type definition and its instances
- Each instance has
  - **Unique object identifier (OID)** ensures that all entities included in the database can be unambiguously identified
  - **State** – a set of values contained by object. Values represent properties of the entity, including facts about relationships with other entities.
  - **Behaviour** – possible actions or methods that can be applied to an object. Actions describe how the entity can be used.

Oracle object type (1/3)

- Similar to C++ or Java classes
- In ORDB each object is an instance of some object type
- Oracle object type has
  - Specification or definition part
    - Specifications of attributes
    - Definitions of method interfaces and calls
  - Body that contains program code

Oracle object type(2/3)

- **Object type contains the following characteristics:**
  - One or more attributes, that define structure of the object
  - None or some methods that implement the behaviour of the object



Oracle object type(3/3)

- Object type is one kind of data types
  - Variable of object type is used to contain the value of this type
  - It can be used in the same way as any other type like NUMBER or VARCHAR2
  - Object type can be used as a data type of some column in relational table
  - Variable of object type can be used in PL/SQL

- The value of an object type is an **instance** of the corresponding type. An instance is also called **object**
  - Value can be assigned to the instance's attributes
  - Methods of the instance can be called

Creating object types

- Consists of two steps:
  - Type definition

```
CREATE TYPE  <type_name> AS Object(
<attribute>              <type>,
<attribute>              <type>,
<...>
<definitions of methods interfaces>
...);
```

  - Type body definition

```
CREATE  OR  REPLACE  TYPE BODY schema.type  IS  or  AS
   MAP or ORDER MEMBER function_definition,
   STATIC or MEMBER  function_definition,
   STATIC or MEMBER  function_definition …
   STATIC or MEMBER  procedure_definition,
   STATIC or MEMBER  procedure_definition,  …
END;
```

Type definition example

```
CREATE  OR REPLACE  TYPE  Object_Employee  AS  OBJECT  (
   ID          NUMBER,
   NAME           VARCHAR (25),
   SURNAME        VARCHAR (35),
   SALARY         NUMBER (5,2),
   MEMBER  PROCEDURE  Print_Employee );

Type created.
```

Oracle system of data types

- Oracle object type system is realized as an extension of relational model
- Object type interface supports standard functions of relational database
  - Like queries (*Select…From…Where*)
- SQL and programming interfaces have been extended to support objects

SQL extensions

- Type inheritance
- Type evolution
- Object views
- SQL object extensions
- PL/SQL object extensions
- Java support for Oracle objects
- External procedures
- Etc.

24

Type body creation example

```
CREATE  OR  REPLACE  TYPE  BODY  Object_Employee  AS
   MEMBER  PROCEDURE  Print_Employee  IS
     BEGIN
       DBMS_OUTPUT.PUT('Employee ID number: '|| ID );
       DBMS_OUTPUT.PUT(' Name: '||NAME||' Surname: '|| SURNAME);
       DBMS_OUTPUT.PUT(' Salary: '|| SALARY);
       DBMS_OUTPUT.NEW_LINE;
     END  Print_Employee;
END;

Type body created.
```

Creating instances

```
DECLARE
   VARIABLE TYPE;
```

For example,

```
DECLARE
     STUDENT_INSTANCE STUDENTS;
     …
```

- According to the syntax of the PL/SQL language object declared in such a way is initialized as a NULL object
  - It is not possible to refer values of its attributes
  - To start operating with its attributes we have to initialize the object by using specific functions – constructors

Initializing object instances

```
DECLARE
   STUDENT_INSTANCE : = STUDENT(111, 'Jānis', 'Kalniņš', '001RDB001,
   '11118155555',
   'Rīga, Kr.Valdemāra iela 111, dz.21', '67333333', 11);

BEGIN
   STUDENT_INSTANCE.PHONE  : = '674444444';
. . .
```


27

Principal idea of the ORDBS – object types and storage structures

Object storage

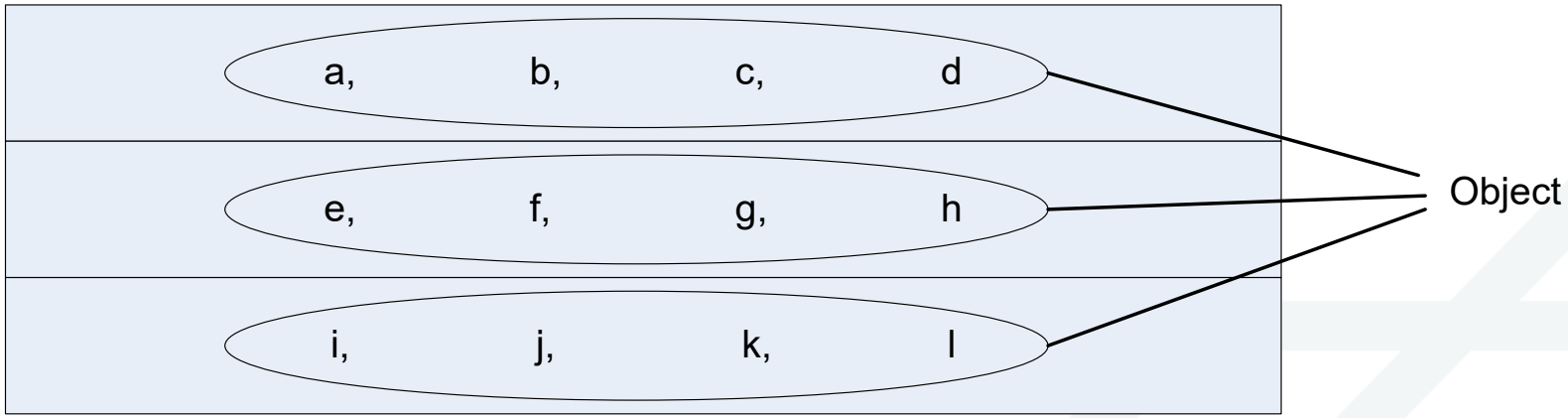
Main data storage structures of ORDB

- **Table with row type objects**
- **Table with object columns**
- **Table with object collection**
- Table with row type objects and object collection
- **Table with array of objects**
- Table with different type row objects

Object tables

Object tables

- Each row is an object
- In comparison to RDB
  - Objects instead of rows
  - Attributes instead of columns



Object tables

- Objects that occupy the whole row of the object table are called **row objects**
- All statements that can be used with the relational table, can also be used with object tables
- All integrity constraints and trigers supported by relational tables are supported also by object tables

Creating object tables

- Object table PERSON:

```
CREATE TYPE PERSON AS OBJECT(
name        varchar2(20),
surname     varchar2(30),
phone       varchar2(20));

CREATE TABLE PEOPLE OF PERSON;
```

Adding constraints

```
CREATE TABLE PEOPLE OF
PERSON
(
phone PRIMARY KEY
);
```

Inserting data into object tables (1/4)

Data can be inserted in object tables in three ways:

1. Insertion with operator INSERT and constructor function
2. Data insertion into table using a program
3. Data insertion using operator INSERT and without the constructor function*

*Can be used if some insertions into the table already have been done

Inserting data into object tables (2/4)

Insertion with operator INSERT and constructor function can be used in the following way:

```
INSERT  INTO  PEOPLE  VALUES
   (PERSON('Jane', 'Smith', 2999999));
```

Inserting data into object tables(3/4)

Data insertion into table using a program can be done as follows:

```
DECLARE    Variable_1  PERSON;

BEGIN

Variable_1 := PERSON( Phone => 25858585, name => 'John', surname
  => 'Smith');
INSERT  INTO  PEOPLE  VALUES (Variable_1);

END;

PL/SQL procedure successfully completed.
```

Inserting data into object tables (4/4)

Data insertion using operator INSERT and without the constructor function *

*Can be used if some insertions into the table have already been done

```
INSERT  INTO  PEOPLE  VALUES  ('Cris', 'Robinson', 2999990);
1 row created.
```

Retrieving data from object tables (1/3)

Data can be retrieved from object tables in two ways:

1. As a table consisting of one column, where each row is of type Person, that allows execution of object oriented operations
   - Use operator VALUE
   - Result is relational table with one column, whose type is PERSON

2. As a relational table having multiple columns. Each attribute of the object Person occupies one column. Such a table allows to execute all relational operations
   - Do not use operator VALUE
   - The query returns relational table according to the higher level attributes of the object

Retrieving data from object tables(2/3)

1. Object oriented data retrieval

```
SELECT  VALUE(A)  FROM  PEOPLE  A;
```

Any selection criteria can be added:

```
SELECT   VALUE(A)
     FROM       PEOPLE  A
     WHERE    A.name='Jane';
```

Retrieving data from object tables (3/3)

2. Data retrieval as from a relational table:

```
SELECT A.NAME, A.SURNAME FROM PEOPLE A;


SELECT * FROM PEOPLE;
```

Changing state of an object

- Objects allow executing other operations similarly to the relational tables
- State of the object can be modified by the SQL operator UPDATE:

```
UPDATE  PERSONS  A
SET  A = PERSON('Anita', 'Koks', '3333333')
WHERE  A.surname='Koks' and A.name='Anita';
```

Object columns

Object columns

- Object tables is not the only way to store objects
- Each field of the relational table and each attribute of an object can be of object type
- Such objects that are stored as columns or attributes are named **column objects** or field objects

Creating object columns

- We will use the previously defined type Person
- One of the columns will have this type

```
CREATE  TABLE EMPLOYEES(
Id    number(4) Primary key,
employee     PERSON,
position     varchar2(30),
salary       number(6,2));
```

Data insertion into table with object column

- INSERT operator is used together with constructor function to create an object for the column employee:

```
INSERT INTO Employees VALUES (101, Person('John', 'Smith','1111111'),
  'Bookkeeper', 180.00);
INSERT INTO Employees VALUES (102, Person('Jane', 'Smith', '2222222'),
  'CEO', 200.00);
```

Retrieving data from tables with object columns (1/2)

- Similarly to the object tables it is possible to retrieve whole objects and separate attributes
- By retrieving all data we will get the following result:

```
SELECT * FROM Employees;
```


48

Retrieving data from tables with object columns (2/2)

- By retrieving only the object column we will get the following result:

```
SELECT A.EMPLOYEE  FROM  EMPLOYEES  A;
```

- It is also possible to retrieve only one or some attributes of the object, for example name of the employee:

```
SELECT A.EMPLOYEE.NAME FROM Employees A;
```

Changing attributes of objects

Object attributes can be changed with an update query:

```
UPDATE Employees A
SET A.employee.surname='Robinson'
WHERE id=101;
```

NULL objects

- Object whose value is NULL is an atomic NULL
  - Differs from the object, whose all attributes are NULL
  - Can not be changed (attributes can not be accessed)
  - Methods can not be called
  - In fact the whole object does not exist

- When inserting data
  - Any column object may be NULL
  - Row objects can not be NULL

Object Collections

Collections

- In RDB relationship one to many was stored in two tables
- ORDB allows to store such a relationship in 1 table
- It is done by using collections
- Two types of collections exist:
  - *Varying array - varray*
  - *Nested table*

Varray

- Varying array or varray is an ordered and limited set of elements
- It is an order of none or some elements with the same type
- Each element has a position or index
  - Uniquely identifies an element
  - Identifies object's position in the array
    - Position is an integer in the range from 1 to the defined maximal position of the array

Creating Varrays

- During the creation of such a type system automatically defines its constructor
  - Constructor is used to create VARRAY instances
- VARRAY can not contain LOB objects
  - VARRAY can not contain objects with attributes of type LOB
- COUNT is built in attribute of the VARRAY that returns the total number of elements
- We will create STUDENT type that has one field that contains all 3 grades:

```
CREATE TYPE GRADE_TYPE AS VARRAY(3) OF number(2);
```

Retrieving data from VARRAYS

- Oracle allows to execute the following queries with VARRAYS:

```
SELECT * FROM STUDENTS;
```

- It is possible to retrieve separate elements from the VARRAY:

```
SELECT C.* FROM STUDENTS A, TABLE(A.GRADES) C  WHERE A.ID =
  1;
```

Nested tables (1/3)

- Nested table is an unlimited and unsorted set of elements
- All elements have the same data type
- There is no need to define maximal number of elements
- It is possible to insert, update and delete records in the same way as with the relational table
- Elements of the nested table are physically stored in separate table that identifies the record of the main table or object to whom the element belongs

57

Nested tables (2/3)

- Nested table has a **single column** and its type is either:
  - Built in Oracle data type or
  - Object type

58

Nested tables (3/3)

Tabule_Educational_Books

Speciality	Student_Course	List of the literature
Databases	102	
Information systems	201	
Design	103	

Table_Books

Title	Author	Number
Databases	J. Koks	123
Information systems	Z.Sakne	124
DB management systems	L.Lapa	125
DB technologies	I. Ozols	126
DB design	R. Smilga	127

59

Creating nested table

The following steps must be done to create nested table

1. Create object type
2. Create nested table, whose records are of the previously defined object type
3. Create the main table and include the nested table into its definition
4. Insert data

60

Creating nested table. Example (1/3)

1. Create the object type:

```
CREATE  TYPE  Object_type_Book  AS  OBJECT (
   Title       VARCHAR2(50),
   Author      VARCHAR2(30),
   Num         NUMBER(6));
Type created.
```

2. Create nested table, whose records are of the previously defined object type

```
CREATE  TYPE  Table_Books  AS  TABLE  OF  Object_Type_Book;
Type created.
```

Creating nested table. Example (2/3)

3. Create the main table and include the nested table into its definition

```
CREATE  TABLE  Table_Educational_Books  (
 Speciality                VARCHAR2(20),
 Student_Course            NUMBER(3),
 Literature_List           Table_Books)
   NESTED  TABLE Literature_List STORE  AS
   Nested_Table_Books;
Table created.
```

Creating nested table. Example (3/3)

4. Insert data

```
INSERT  INTO  Table_Educational_Books  VALUES  (‘Databases',
  102,
     Table_Books(
Object_Type_Book(Databases', 'J.Koks', 123),
Object_Type_Book('DB technologies ', 'I.Ozols', 126)
)
);
```

Queries for nested tables

- There are two ways how to query the table that contains column with a collection type
  - Put the collections into the rows that contain them
  - Create a corresponding row for each collection type

- It is also possible to retrieve data using PL/SQL program and cursor

Returning collection as a nested result

- Example: column *Projects* is a nested collection with a type List_Of_Projects

```
SELECT Literature_List
FROM Table_Educational_Books;
```

Spreading out the collection (1)

- Not all tools and applications are capable to cope with the nested results
  - There may be a need to retrieve pure relational structure by spreading the collection into some relational rows
- It is done by using TABLE expression together with the collection
  - The table expression is included in the FROM expression as a table
  - TABLE expression takes the role of the subquery

```
Select b.Speciality, p.*
From Table_Educational_Books b,
  Table(b.Literature_List) p;
```

Spreading out the collection (2)

- If there is a need to retrieve employees that have no projects:

```
Select e.*, p.*
From employees e, Table (e.list_of_projects)(+) p;
```

(+) means, that the result will contain records that have no elements in the nested collection or list itself is null. Such records will have NULL value in the corresponding resulting column

67

Choice between VARRAY and nested table

- VARRAY is more suitable if
  - Number of elements is fixed and known
  - There is a need to move between elements in some order
  - Retrieve and manipulate the whole collection as a single value

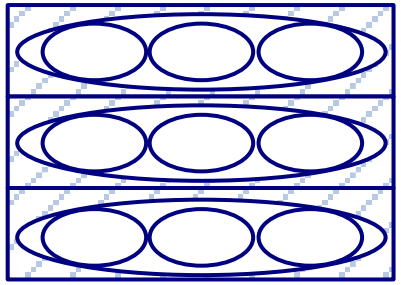
- Nested table is more suitable if we need
  - To effectively execute queries with the collection
  - Store large number of elements
  - Execute many insert, update and delete operations

Multi level collections

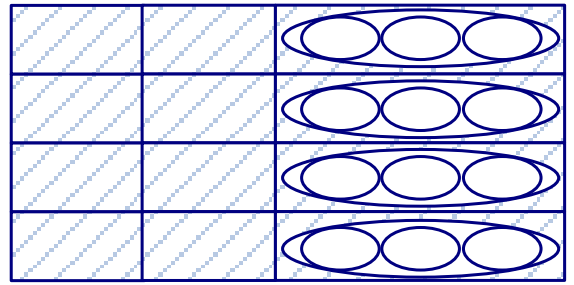
- Collections can be created in multiple hierarchical levels
  - Elements of one collection can be another collection
- Such collections are named a multi level collection types
  - Collection types whose attributes directly or indirectly contain collection types
- There are several options how to build the collection type:
  - Nested table of the nested table type
  - Nested table of the VARRAY type
  - VARRAY of the nested table type
  - VARRAY of the VARRAY type
  - Nested table or VARRAY with an object type whose attribute is a VARRAY or nested table type

To sum up…

- Row type objects (object table)



- Table with object column



- Table with object collection